MLOps Lab Assignment 4–5

Advanced MLflow and Data Versioning with DVC

Reproducible House-Price Regression Workflow

Student	Hemant Malviya
Programme	B.Tech — Artificial Intelligence & Data Science
Institution	Gati Shakti Vishwavidyalaya
Tools	Git · DVC · MLflow · scikit-learn
GitHub repository	vixvo777/ML_HousePrice_prediction
Submission branch	mlops-lab-submission

This report includes project methodology, measured model results, registry lifecycle evidence, dataset version comparison, and screenshots captured during the lab.

1. Objectives

The objective is to demonstrate a reproducible machine-learning workflow in which an engineer can inspect experiment results, identify a registered model version, connect a run to code/data lineage, and restore a previous dataset version.

• Track and compare Linear Regression, Ridge Regression, and Random Forest runs with MLflow.

• Record evaluation metrics, model artifacts, and model signature/input example where available.

• Register model versions and document lifecycle metadata.

• Version a CSV dataset with DVC while Git tracks DVC metadata and history.

• Verify restoration of the baseline dataset and preserve the current dataset version.

2. Environment and project setup

Component	Configuration / evidence
OS / terminal	Windows PowerShell in VS Code
Python	3.13.4
MLflow	3.17.0
DVC	3.67.1
Dataset	House-price regression CSV; 6 columns including target `price`
Git local branch	master
GitHub submission branch	mlops-lab-submission
DVC remote	`localremote` — local directory outside the repository

Repository: https://github.com/vixvo777/ML_HousePrice_prediction/tree/mlops-lab-submission

Setup and reproduction

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
python src/train.py
```

The MLflow UI can be started with `mlflow ui --host 127.0.0.1 --port 5000`. Open http://127.0.0.1:5000. Git and DVC are already initialized in the existing repository; do not initialize them again in a checkout.

3. MLflow experiment tracking

The experiment `HousePricePrediction` compares three regression candidates. MAE and RMSE measure prediction error (lower is better); R^2 indicates explained variance (higher is generally better).

Run	MAE	RMSE	R^2
LinearRegression	37,179.6346	46,062.3006	0.9872475
Ridge_alpha_1	37,203.8322	46,068.3859	0.9872441
RandomForest_100	106,252.7815	144,660.8478	0.8742216

The values above are transcribed from the provided `reports/model_comparison.csv` screenshot. Linear Regression has the lowest MAE and RMSE and the highest R^2 among the three captured runs. This is the strongest model by these displayed test metrics.

Experiment runs and model comparison evidence

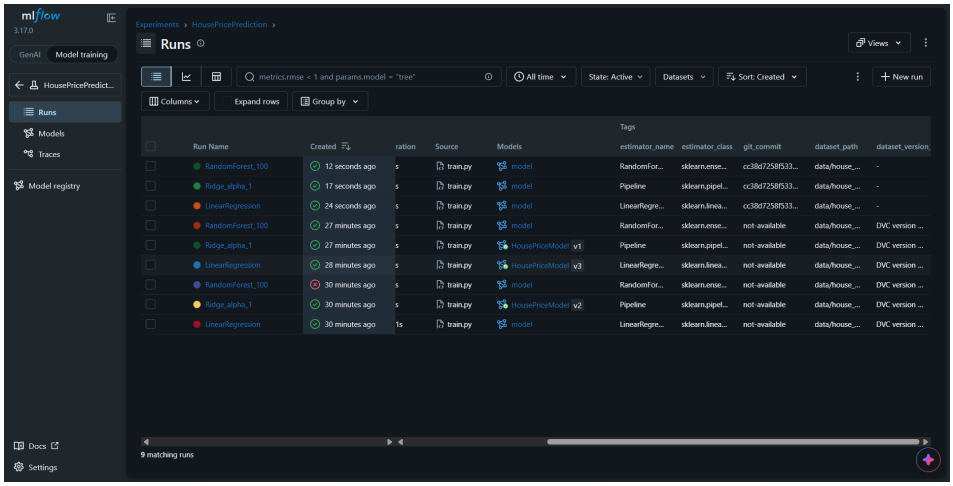


Figure 1. MLflow experiment run list showing candidate runs and their metrics.

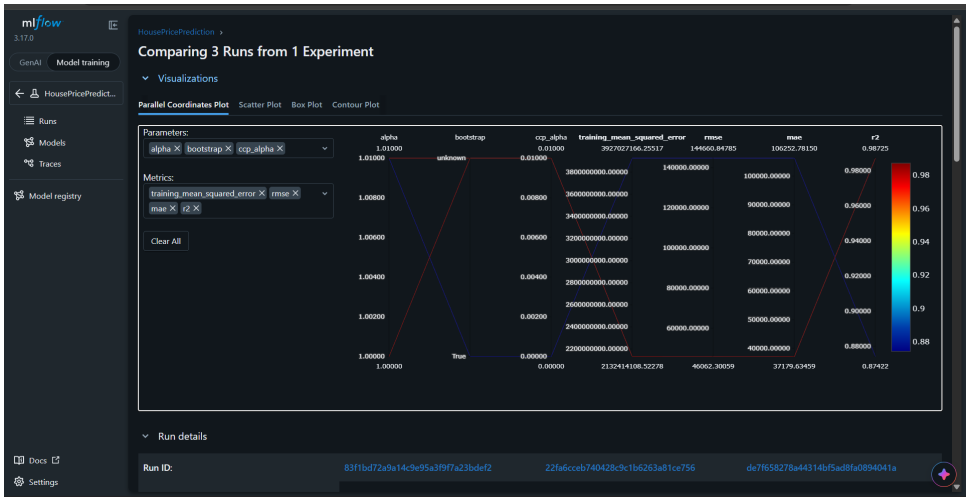


Figure 2. MLflow model/run comparison view.

4. Model artifacts and evaluation

MLflow associates each run with evaluation metrics and artifacts. The captured Random Forest run includes an actual-versus-predicted figure, a JSON metrics report, and model information. A model signature describes expected input/output schema; an input example demonstrates the intended input structure. Autologging captures supported details depending on the installed flavor and training configuration.

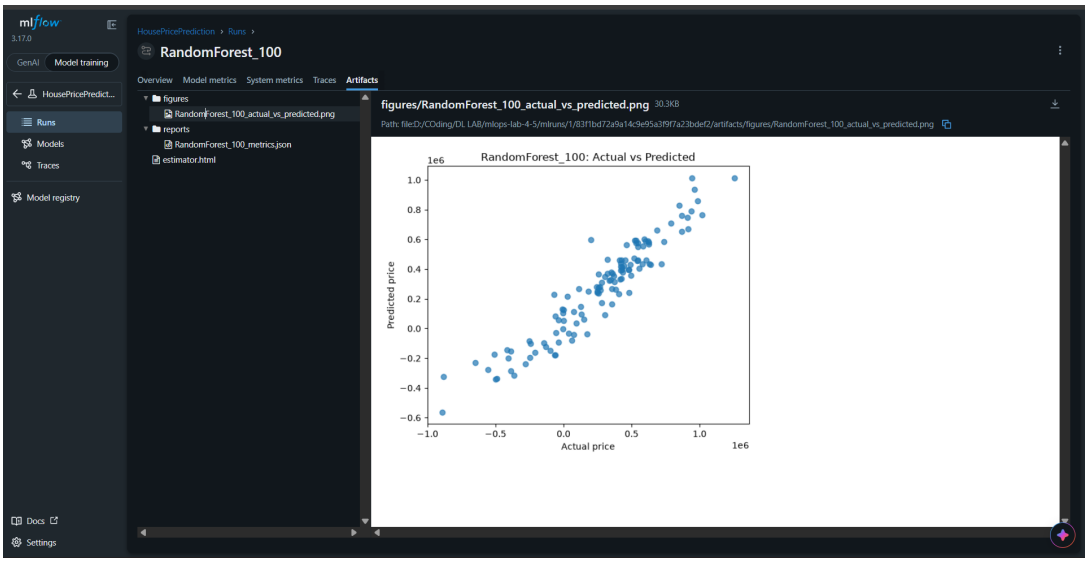


Figure 3. Random Forest actual-versus-predicted plot opened from the MLflow run artifacts.

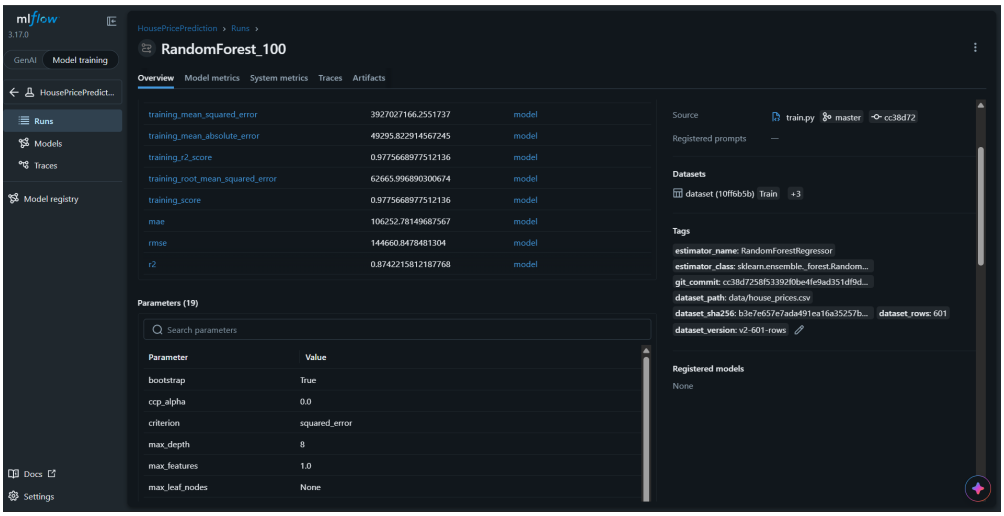


Figure 4. RandomForest_100 run details and logged metrics/artifacts.

5. Model Registry and lifecycle

The registered model is `HousePriceModel`. The captured registry evidence shows three versions. Version 2 has the `production` alias and `lifecycle: production` tag; Versions 1 and 3 are tagged retired. Lifecycle tags/aliases describe registry intent and do not by themselves prove deployment to a live serving endpoint.

Version	Observed lifecycle	Interpretation
1	Retired	Earlier registered version retained as history
2	Production alias; lifecycle: production	Version designated for production in captured registry evidence
3	Retired	Alternative registered version retained as history

The captured metric table favors Linear Regression, whereas the registry screenshot designates Version 2 for production. Before submission, confirm which run/model is linked to Version 2 and describe the actual promotion rationale; do not assume Version 2 is the Random Forest model.

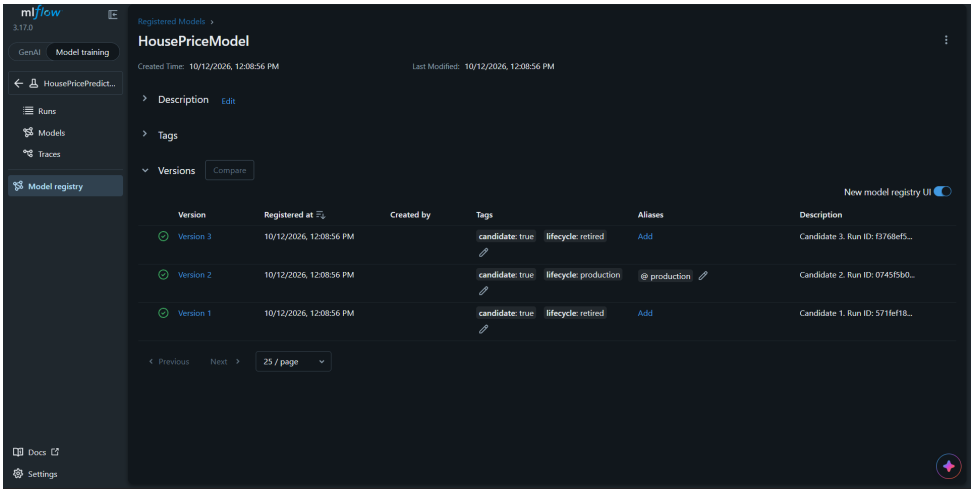


Figure 5. Registered model detail view.

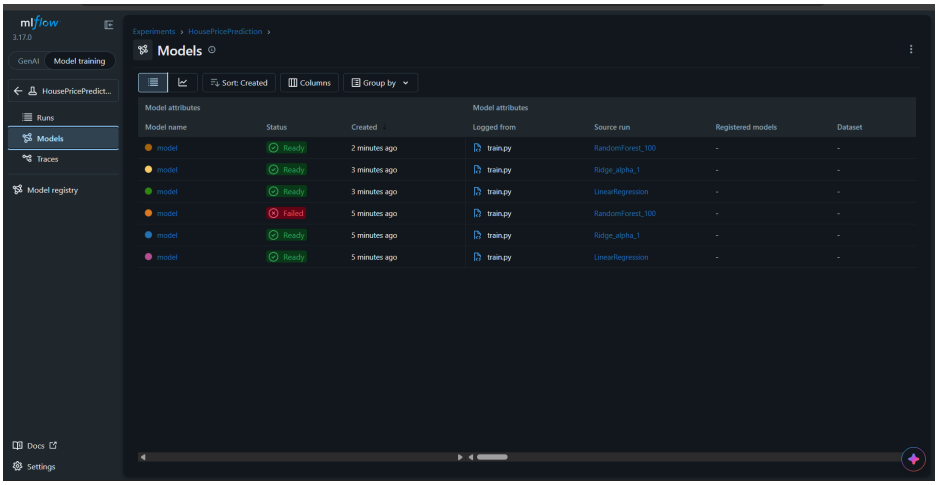


Figure 6. Model registry versions and lifecycle metadata.

6. DVC dataset versioning

DVC stores data objects in its cache/remote while Git versions the `.dvc` metadata. The metadata records a content hash and file size, allowing a particular dataset version to be identified and recovered.

Version	Git commit	Rows	Columns	Recovery status
Baseline	db35164	600	6	Restored in separate worktree; row count verified
Version 2	0333e99	601	6	Main project verified; `dvc status` up to date

Baseline MD5: `befa2e8b79b899ef5647af5d90a0d9ae` (65,591 bytes). Version 2 MD5: `6c20cf1de1a57eb24b7634fad3bd5b2f` (64,649 bytes).

Baseline recovery evidence



Figure 7. Successful DVC pull/checkout and pandas verification showing 600 rows and 6 columns.

Latest dataset retained in main project

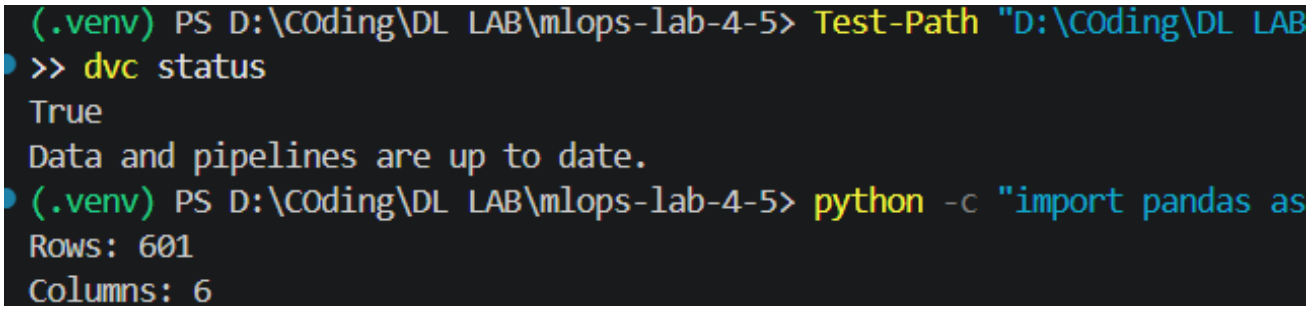


Figure 8. Main project remains at 601 rows and 6 columns, with DVC status up to date.

Recovery was initially blocked because the relative DVC remote path resolved differently in the separate worktree. Changing the recovery worktree remote URL to `D:\COding\DL LAB\dvc-storage` allowed `dvc pull` to fetch the baseline and `dvc checkout` to apply it. The original project was not overwritten.

7. Git, DVC, and MLflow integration

Tool	Tracks	Role
Git	Code, configuration, DVC metadata history	Identifies source revision and historical dataset metadata
DVC	Dataset content hashes, cache and remote objects	Restores the selected data version when objects are available
MLflow	Runs, parameters, metrics, artifacts, registered versions and aliases	Compares candidates and records model/run lineage

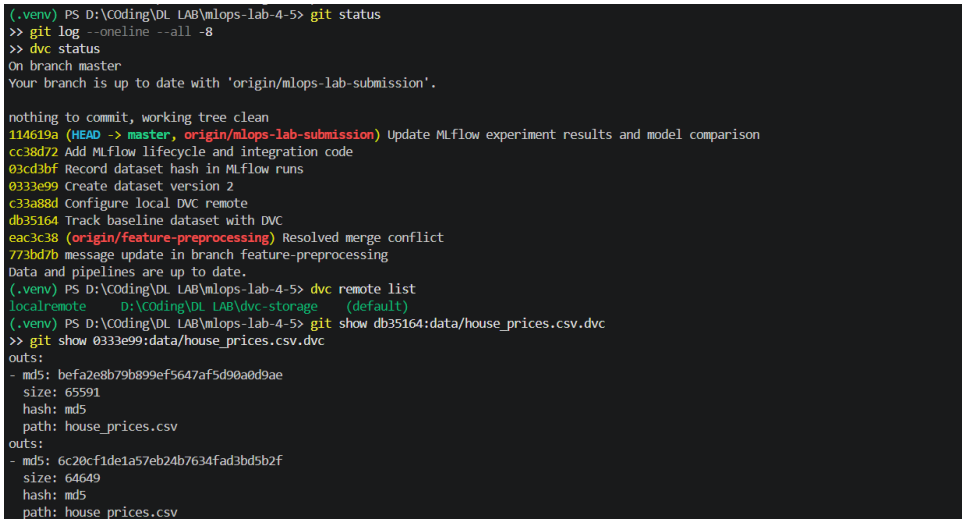


Figure 9. Git history and DVC status showing baseline, remote configuration, and dataset Version 2 commits.

The workflow links source code revision, dataset identity, MLflow run metrics, and registered model metadata. The local DVC remote is not shared cloud storage; another engineer would need access to the data objects or a shared remote to recover the dataset on another machine.

8. Challenges and resolutions

DVC remote path

The recovery worktree resolved a relative path incorrectly. Setting the explicit local remote path fixed retrieval.

Dataset recovery

DVC successfully restored the 600-row baseline in a separate worktree; the main project retained the 601-row version.

Git lineage

After the project was committed, later MLflow runs recorded the Git commit identifier rather than `not-available`.

Model serialization

A Random Forest serialization issue involved the scikit-learn tree type; the training workflow was adjusted to trust the specific locally trained `sklearn.tree._tree.Tree` type.

9. Conclusion

The lab demonstrates MLflow experiment tracking and registry metadata, DVC dataset versioning, and Git history integration. Three candidates were compared; Linear Regression had the strongest displayed test metrics. The registry evidence marks Version 2 for production and Versions 1 and 3 retired, so the promotion rationale should be explicitly tied to the model/run registered as Version 2. Dataset recovery was verified: the baseline was restored to 600 rows in a separate worktree, while the main project remained at 601 rows.

The main reproducibility limitation is that the DVC remote is local to the machine. A shared remote is needed for another engineer to recover the dataset from a separate computer.

10. References

• MLflow documentation: https://mlflow.org/docs/latest/

• DVC documentation: https://dvc.org/doc

• scikit-learn documentation: https://scikit-learn.org/stable/

• MLOps Lab Assignment 4–5: Advanced MLflow and Data Versioning with DVC (course handout).

Submission evidence checklist

■ Embed/retain MLflow run, artifact, and registry screenshots (Figures 1–6).

■ Include the successful DVC recovery screenshot and main-project 601-row verification (Figures 7–8).

■ Include Git/DVC history evidence (Figure 9).

■ Confirm the run/model corresponding to registry Version 2 and add its promotion rationale.

■ Submit this PDF, GitHub repository URL/branch, README.md, requirements.txt, and supporting screenshots if requested separately.